MarketSage AI

Master Engineering Blueprint — **Part I**

System Architecture and Data Layer

Nine-plane architecture, two speed lanes, technology choices, source catalogue, candle builder, point-in-time discipline, storage schema and data quality gates.

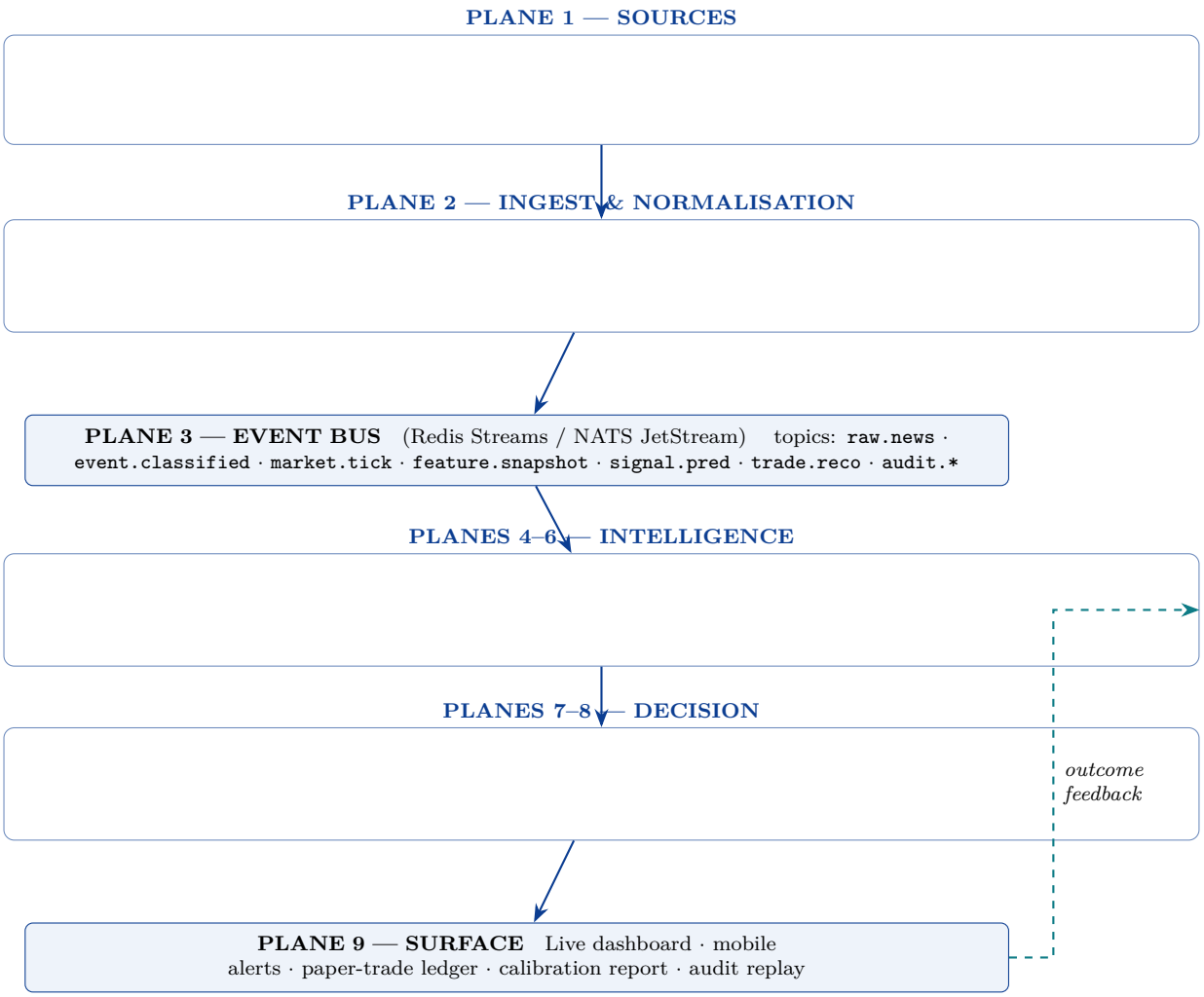
This is an extract of the MarketSage AI master plan, published separately for convenience. Chapter, section and equation numbering matches the full document, and cross-references point back into it. Read Chapter 3 (Reality Calibration) first — every design decision in this part follows from it.

CHAPTER 4

System Architecture

4.1 The nine-plane view

The system is decomposed into nine planes. Each plane is independently deployable, independently testable, and communicates only through the event bus with versioned schemas. A plane may never reach into another plane's database.



4.2 Plane responsibilities and contracts

Plane	Owns	Hard contract it must honour
1 Sources	Nothing (external)	Every item must carry an original publish timestamp or be flagged `ts_uncertain`.
2 Ingest	Collectors, normaliser, dedup, candle builder	Emits only immutable, schema-versioned records. Never mutates history. Idempotent on replay.

Plane	Owns	Hard contract it must honour
3 Bus	Ordering, durability, replay	At-least-once delivery, consumer groups, 7-day retention hot, cold archive to Parquet.
4 Event	Taxonomy classification, entity extraction, novelty, credibility	Emits an `Event` with class, severity, confidence, entities, novelty and credibility. Never emits a trade opinion.
5 Impact	Historical conditional response distributions	Emits a distribution over index returns per horizon, plus sample size and confidence interval. Refuses to answer when $n < 8$.
6 Features	Online and offline feature computation	*Identical code path* for training and serving. Every feature carries an as-of timestamp.
7 Prediction	Direction, magnitude, volatility, quantiles	Outputs calibrated probabilities and conformal intervals. Must attach model version and feature hash.
8 Decision	Structure choice, sizing, risk	May veto anything. Owns all kill-switches. Writes an immutable decision record with full reasoning.
9 Surface	Presentation, alerting, ledger	Read-only with respect to trading state; may not influence decisions.

4.3 Two speed lanes

A recurring failure in such systems is trying to make everything fast. Instead, split explicitly:

	Hot lane (reflex)	Cold lane (deliberation)
Latency budget	Under 400 ms end to end	1–60 seconds
Triggered by	Breaking event, price shock, scheduled release	Every event, plus a 5-second heartbeat
Models used	Quantised, pre-warmed, small; rules and lookups	Full ensemble, LLM reasoning, cross-source confirmation, Monte Carlo
Output	Provisional alert with wide intervals	Full recommendation with structure and sizing
Can it place an order?	No. Never.	Yes (once Tier T4 is reached)
Rationale	Speed without confirmation is how you trade a fake headline	Confirmation without speed misses the move; two lanes get both

The two-lane rule

The hot lane says *"something is happening, widen your risk"*. The cold lane says *"here is the trade"*. Conflating them is the design error that turns a research system into an account-destroying machine.

4.4 Technology choices with justification

Layer	Choice	Why this, not the alternative
Language (services)	Python 3.12 + `asyncio` + `uvloop`	Ecosystem for ML is decisive. `uvloop` closes most of the gap for I/O bound work. Rewrite only the proven hot path later, if profiling demands it.
Language (hot path, later)	Rust (via PyO3)	Only for the tick handler and candle builder if p99 exceeds budget. Do not start here.
API	FastAPI	Async, typed, automatic OpenAPI, WebSocket support.
Event bus	Redis Streams (v1) → NATS JetStream (v2)	Redis Streams gives consumer groups and replay with zero extra infrastructure since Redis is already needed for the online feature store. Kafka is operationally heavy for a single-node start; NATS is the low-latency upgrade path.
Time-series store	PostgreSQL + TimescaleDB	Hypertables, continuous aggregates for candle roll-ups, compression, and plain SQL. Avoids a separate query language.
Research store	Parquet on local NVMe + DuckDB	Backtests are analytical scans, not point lookups. DuckDB over Parquet is dramatically faster and simpler than querying the OLTP database.
Online feature store	Redis (hashes + sorted sets)	Sub-millisecond reads, TTL semantics match feature freshness.
Vector search	FAISS or `pgvector`	Deduplication and nearest-historical-event retrieval. `pgvector` first for operational simplicity.
Model training	PyTorch, LightGBM, scikit-learn	LightGBM is the baseline that usually wins on tabular features; PyTorch for sequence and distribution models.
Model serving	ONNX Runtime	Removes Python overhead at inference; consistent across CPU and GPU; batch-1 latency is what matters here.
NLP models	A financial-domain encoder for sentiment/classification, a multilingual encoder for global sources, plus an LLM in the cold lane only	Encoders are fast and cheap; the LLM is reserved for reasoning where its latency is affordable.
Orchestration	Prefect or Dagster	Nightly retrains, data quality checks, backfills. Airflow is heavier than needed.
Experiment tracking	MLflow	Model registry plus metric lineage; the registry is what production reads from.
Frontend	React + TypeScript + a lightweight charting library	TradingView's free charting library is acceptable for candles; the fan-chart overlay is custom canvas work.
Transport to UI	WebSocket with binary framing (MessagePack)	JSON at 1 Hz per instrument is fine; at tick rate it is not.

Containerisation	Docker Compose (v1) → Kubernetes (only at multi-user)	Kubernetes before Tier T5 is pure overhead.
Monitoring	Prometheus + Grafana + structured JSON logs + OpenTelemetry traces	A trace ID must flow from the news item all the way to the recommendation.

Deployment location matters more than framework choice

Co-locate compute in the same cloud region as your broker/data endpoints (for NSE-facing infrastructure this generally means a Mumbai region). A poor region choice adds more latency than every micro-optimisation in this document combined.

4.5 Failure domains and degradation

Failure	Detection	Degraded behaviour
News source down	No heartbeat for 90 s	Mark source stale; reduce credibility weight of single-source events; continue on market-only features.
Market feed gap	Sequence gap or 3 s silence during market hours	Halt all new signals immediately. Flatten open paper positions at last known price. Alert.
Model server unhealthy	p99 latency or error rate breach	Fall back to the rules-only baseline model, flag every output as `DEGRADED`, forbid auto-execution.
Clock drift > 50 ms	NTP monitor	Refuse to emit signals. Time integrity is non-negotiable for point-in-time correctness.
Redis unavailable	Connection error	Hot lane stops; cold lane continues from TimescaleDB with a wider latency SLO.
Contradictory events	Two opposing high-severity events within 60 s	Emit `CONFLICT`; suppress all directional signals; volatility signals only.

CHAPTER 5

Data Layer

5.1 Source catalogue

Sources are tiered by credibility. Credibility is a numeric weight used later in impact scoring, not a binary trust decision.

Source class	Credibility c	Typical lag	Role
Central bank / regulator official release	1.00	0–2 s	Ground truth for policy events. Scrape the official page directly, not a news summary of it.
Exchange notices	1.00	0–5 s	Halts, bans, margin changes.
Major international wires	0.90	1–10 s	Primary breaking coverage.
Verified official accounts of heads of state / ministries	0.85	0–3 s	Highest-impact, highest-noise. Requires source-specific handling.
Tier-1 national business media	0.75	30–180 s	Confirmation, colour.
Aggregator APIs	0.60	60–600 s	Breadth, not speed.
Anonymous social accounts	0.15–0.35	0–5 s	Early warning only. May *never* independently trigger a directional trade.
Forums and community feeds	0.10	Variable	Sentiment aggregate only.

5.1.1 Market data

Stream	Frequency	Use
Index spot (Nifty, Bank Nifty, Fin Nifty)	Tick or 1/s	Base price series, candle construction.
Index futures	Tick or 1/s	Basis, and it often leads spot.
Full option chain (both indices, near two expiries)	1–5 s snapshot	Implied volatility surface, open interest, put-call ratio, max pain, gamma exposure.
India VIX	1/s	Regime variable, volatility term structure anchor.
GIFT/SGX Nifty	Tick	Overnight and pre-open leading indicator.
Global futures (US equity index, crude, gold), DXY, USDINR	1/s or 1-min	Cross-asset propagation, overnight event pricing.
Sector indices	1/s	Event-to-sector routing.
Participant-wise open interest, FII/DII flows	End of day	Positioning regime features.

Corporate actions, expiry calendar, holiday calendar	Static/daily	Absolutely required to avoid silent backtest corruption.

5.2 The candle builder (and why it is subtle)

Building 1-minute candles from ticks looks trivial and is the most common source of quiet data corruption. Requirements:

1. **Exchange time, not local time.** Every timestamp normalised to exchange time with explicit timezone; store as UTC with an exchange-time view.
2. **Left-closed, right-open intervals.** A tick at exactly 09:20:00.000 belongs to the 09:20 candle, not 09:19.
3. **Explicit gap handling.** If no tick arrives in a minute, emit a zero-volume candle with $O = H = L = C =$ previous close, flagged `synthetic`. Silent omission destroys sequence models.
4. **Session boundaries.** Pre-open (09:00–09:15) is a different price formation mechanism; tag it, never mix it into continuous-session statistics.
5. **Late ticks.** A tick arriving after its candle is sealed goes to a correction log; the candle is versioned, not overwritten in place.
6. **Determinism.** Replaying the same tick file must yield a byte-identical candle file. Enforce this with a test.

5.3 Point-in-time discipline

The three timestamps

Every record carries three timestamps, always:

- `t_event` — when the thing happened in the world (publish time).
- `t_ingest` — when our system first saw it.
- `t_available` — when it became usable for a decision (after dedup, classification and feature computation).

Backtests may only use data where `t_available` $\leq$ the decision time. A feature computed from `t_event` alone is look-ahead bias wearing a disguise.

Revised data (an economic figure corrected an hour later, an option chain snapshot re-published) is stored as a new row with a validity interval, never as an update. The research layer always reads "the value as known at time t".

5.4 Storage layout

Listing 5.1: Core TimescaleDB schema (abridged)

```
-- Ticks: hypertable partitioned by day, compressed after 7 days
CREATE TABLE tick (
  ts TIMESTAMPTZ NOT NULL,
  instrument_id INT NOT NULL,
  ltp NUMERIC(12,2) NOT NULL,
  qty INT,
  bid NUMERIC(12,2), ask NUMERIC(12,2),
  bid_qty INT, ask_qty INT,
  oi BIGINT,
```

```
  seq BIGINT NOT NULL, -- gap detection
  src SMALLINT NOT NULL
);
SELECT create_hypertable('tick','ts', chunk_time_interval => INTERVAL '1 day');

-- Candles: one row per (instrument, timeframe, bucket)
CREATE TABLE candle (
  ts TIMESTAMPTZ NOT NULL, instrument_id INT NOT NULL, tf SMALLINT NOT NULL,
  o NUMERIC(12,2), h NUMERIC(12,2), l NUMERIC(12,2), c NUMERIC(12,2),
  v BIGINT, oi BIGINT, trades INT,
  vwap NUMERIC(12,2), synthetic BOOLEAN DEFAULT FALSE, version SMALLINT DEFAULT 1,
  PRIMARY KEY (instrument_id, tf, ts, version)
);

-- Option chain snapshots
CREATE TABLE option_snap (
  ts TIMESTAMPTZ NOT NULL, underlying_id INT, expiry DATE,
  strike NUMERIC(10,2), opt_type CHAR(2),
  ltp NUMERIC(10,2), bid NUMERIC(10,2), ask NUMERIC(10,2),
  volume BIGINT, oi BIGINT, oi_change BIGINT,
  iv NUMERIC(6,3), delta NUMERIC(6,4), gamma NUMERIC(10,8),
  theta NUMERIC(8,4), vega NUMERIC(8,4), rho NUMERIC(8,4),
  spot NUMERIC(12,2)
);

-- News / event store: append-only, point-in-time
CREATE TABLE news_raw (
  id UUID PRIMARY KEY,
  t_event TIMESTAMPTZ, t_ingest TIMESTAMPTZ NOT NULL, ts_uncertain BOOLEAN,
  source_id INT NOT NULL, url TEXT, lang CHAR(3),
  headline TEXT, body TEXT, content_hash BYTEA, simhash BIGINT,
  embedding vector(768)
);

CREATE TABLE event_classified (
  id UUID PRIMARY KEY, news_id UUID REFERENCES news_raw(id),
  t_available TIMESTAMPTZ NOT NULL,
  event_class TEXT NOT NULL, subclass TEXT,
  severity NUMERIC(3,1), confidence NUMERIC(4,3),
  novelty NUMERIC(4,3), credibility NUMERIC(4,3),
  entities JSONB, geo JSONB, surprise NUMERIC(8,4),
  model_version TEXT NOT NULL
);
```

Data	Hot store	Cold store / research
Ticks	TimescaleDB 7 days	Daily Parquet, partitioned `/date=/instrument=`
Candles	TimescaleDB, all	Parquet mirror for DuckDB scans
Option chain	TimescaleDB 30 days	Parquet, partitioned by date and expiry
News & events	PostgreSQL, all	Parquet + embeddings index
Features	Redis (TTL 15 min)	Parquet feature snapshots per decision
Decisions & outcomes	PostgreSQL, all	Never deleted. This is the audit spine.

5.5 Data quality gates (run continuously)

Check	Threshold	Action on breach
Tick sequence gaps	any gap	Log, backfill from broker history, mark affected minute untrusted.

Candle count per session	exactly 375 for continuous session	Block research jobs on that day until resolved.
Option chain completeness	$\geq$ 95% of expected strikes	Do not compute surface; mark surface stale.
Implied volatility sanity	$0.03 < \sigma < 3.0$, no-arbitrage monotonic in strike	Reject the quote, use surface fit instead.
Put-call parity deviation	beyond spread-implied bound	Flag stale quote; do not trade that strike.
News timestamp sanity	`t_event` $\leq$ `t_ingest` and within 7 days	Set `ts_uncertain`, exclude from latency-sensitive research.
Duplicate rate	$< 3\%$ post-dedup	Retune dedup thresholds.
Feature null rate	$< 1\%$ per feature per day	Block model training.
Train/serve skew	feature distribution PSI < 0.2	Alert; investigate before next retrain.